

Laboratorio 2

Contents

1	Introduzione	4
2	Il terminale UNIX	5
2.1	Comandi	5
2.1.1	Struttura base comandi Linux	5
2.1.2	-help e man	5
2.1.3	pwd	5
2.1.4	ls	5
2.1.5	cd	6
2.1.6	mkdir	6
2.1.7	touch	6
2.1.8	stat	7
2.1.9	cp	7
2.1.10	mv	7
2.1.11	rm	7
2.1.12	rmdir	8
2.1.13	cat	8
2.1.14	less	8
2.1.15	head	8
2.1.16	tail	9
2.1.17	wc	9
2.2	Wildcards	9
2.2.1	Asterisco (*)	10
2.2.2	Punto Interrogativo (?)	10
2.3	Reindirizzazione	10
2.3.1	Reindirizzare l'output	10
2.3.2	Reindirizzare l'input	10
2.3.3	Reindirizzare gli errori	10
2.4	Pipe	11
2.5	Virgolette	11
3	Filesystem UNIX	12
3.1	Path	12
3.2	Marcatore temporali	12
3.3	File nascosti	12
4	Programmi UNIX	13
4.1	Flussi standard	13
5	Linguaggio C	14
5.1	Compilatore	14
5.1.1	Warnings	14
5.2	Librerie	14
5.3	Variabili	14
5.4	printf	15
5.4.1	Placeholders	15

5.5	Commenti	15
6	Esercizi	16
6.1	Esercizio 1	16

1 Introduzione

In questo corso lavoreremo con programmi vicini al linguaggio macchina.
Lavoreremo inoltre con:

- UNIX
- thread
- processi
- programmazione di sistema

Codice Teams: r407rxv

2 Il terminale UNIX

Il **terminale** è una interfaccia testuale per dialogare con il sistema.

Dentro il terminale gira una **shell**, ovvero il programma che interpreta i comandi.

Il terminale mostra il **prompt**, riceve i comandi e visualizza l'output.

Il simbolo \$ indica che la shell è pronta a ricevere un comando (potrebbe non essere \$ a volte).

2.1 Comandi

2.1.1 Struttura base comandi Linux

Esiste una struttura base per molti comandi, ovvero questa:

comando + opzione + argomento + (opzione + argomento)

La differenza principale tra opzione e argomento è che l'opzione è relativa alla **modalità** del comando e l'argomento **su cosa lavora** il comando. L'opzione + argomento invece è relativo all'argomento. per esempio:

```
$ gcc -Wall hello.c -o hello
```

2.1.2 -help e man

Entrambi questi comandi aiutano a comprendere specifici comandi in modi diversi, -help è un aiuto generico mentre "man" è il manuale.

```
$ [comando] --help
$ gcc --help
$ man [comando]
$ man gcc
```

2.1.3 pwd

pwd è il comando "print working directory" e mostra la posizione della directory corrente all'interno del **file system**.

```
$ pwd
```

2.1.4 ls

ls, ovvero "list", elenca tutto il contenuto della directory corrente, directory e file.

```
$ ls
```

Può essere usata con alcuni **attributi**, aggiungibili con un "dash" (-) come in questo caso:

```
$ ls -a
```

Ecco gli attributi:

- **a**: include tutti gli elementi nascosti
- **l**: include informazioni avanzate (data, grandezza...)

2.1.5 cd

cd significa "change directory" e si sposta nella directory specificata da un path relativo o assoluto.

```
$ cd [path]
$ cd esercizi
$ cd /users/gabri/esercizi
```

Percorsi speciali:

- **Directory corrente:** specificato da un punto, ha lo stesso valore di `pwd`.

```
$ cd .
```
- **Directory padre:** specificato da un doppio punto.

```
$ cd ..
```
- **Home dell'utente:** specificato da una tilde, si sposta nella home dell'utente ovunque essa sia.

```
$ cd ~
```

2.1.6 mkdir

mkdir significa "make directory" e crea una directory del nome specificato nella cartella corrente.

```
$ mkdir [nomeDirectory]
$ mkdir esercizi
```

2.1.7 touch

Il comando **touch** nasce con l'intento di aggiornare l'ultimo accesso e l'ultima modifica di un file (marcatore temporale, timestamp).

```
$ touch [nomeFile]
$ touch log.txt
```

Se il file non esiste, lo crea.

Tale comando senza attributi modifica:

- Tempo di accesso (`atime`)
- Tempo di modifica (`mtime`)

Se desideriamo poi modificare uno solo di questi timestamp allora si può introdurre un attributo:

- **-a:** `atime`
- **-m:** `mtime`

Il comando può essere usato anche sulle directory, ma non crea automaticamente una directory se assente, bensì solo file.

2.1.8 stat

Il comando **stat**, ovvero "status", mostra i metadati di un file o una directory, inclusi i timestamp.

```
$ stat [nome]
$ stat hello.c
```

Ecco i dati mostrati da tale comando:

- Tipo di file
- Dimensione
- Inode
- Permessi
- Proprietario e Gruppo
- Data di ultimo accesso (Access)
- Data di ultima modifica (Modify)
- Data di ultima modifica dei metadati (Change)

2.1.9 cp

Il comando **cp**, ovvero "copy", copia il file passato come parametro e lo incolla nel path specificato dopo l'argomento, eventualmente rinominandolo.

```
$ cp [nomeFileDaCopiare] [Destinazione]\[NuovoNomeFile]
$ cp hello.c \esercizi\eserc.c
```

Questo esempio copia il file "hello.c", lo sposta nella sub-directory "esercizi" e lo rinomina. Il file originale rimane al suo posto.

La rinomina della copia è omissibile.

Questo comando, con l'aggiunta di un attributo "-r" (**recursive**) può essere applicato anche alle directory.

```
$ cp -r [nomeDirectory] [Destinazione]\[NuovoNomeDirectory]
$ cp -r cartella1 \esercizi\cartella2
```

2.1.10 mv

Il comando **mv**, ovvero "move", sposta o/e rinomina un file:

```
$ mv [nomeVecchio] [destinazione]\[nomeNuovo]
$ mv hello.c ciao.c
```

Questo comando rinomina il file "hello.c" in "ciao.c", senza spostarlo.

2.1.11 rm

Il comando **rm**, ovvero "remove", elimina un file.

```
$ rm [nomeFile]
$ rm hello.c
```

La cancellazione da terminale non sposta nel cestino, è in genere definitiva.

2.1.12 **rmdir**

Il comando **rmdir**, ovvero "remove directory", elimina una directory vuota.

```
$ rmdir [nomeDirectory]
$ rmdir esercizi
```

Se vogliamo eliminare una cartella e il suo contenuto, si possono usare questi attributi:

- **-r**: "recursive", applica la cancellazione anche ai contenuti.
- **-f**: "forced", applica l'attributo recursive con forza, ignora le conferme di cancellazione.

2.1.13 **cat**

Il comando **cat**, ovvero "concatenate", scrive il contenuto del file sullo standard output.

```
$ cat [nomeFile]
$ cat hello.txt
```

Si possono stampare più file in sequenza, concatenati:

```
$ cat [nomeFile1] [nomeFile2]
$ cat hello1.txt hello2.txt
```

Il comando usato da solo diventa interattivo e legge dallo standard input, ovvero il terminale: ogni volta che si scrive qualcosa a cat, lo stamperà direttamente sul terminale.

```
$ cat
$ ciao
ciao
```

Per uscire da questa modalità bisogna premere (**Ctrl + D**).

2.1.14 **less**

Il comando **less**, opera simile al comando **cat**, ma non carica tutto il file nel terminale e per leggere più contenuto bisogna scrollare.

```
$ less [nomeFile]
$ less hello.txt
```

Per uscire bisogna premere **Q**.

2.1.15 **head**

Il comando **head** permette di stampare le prime righe di un file su terminale.

```
$ head [nomeFile]
$ head hello.txt
```

Possiamo specificare quante righe stampare:

```
$ head -n [righeDaStampare] [nomeFile]
$ head -n 5 hello.txt
```

Questo esempio stampa le prime 5 righe del file "hello.txt".

2.1.16 tail

Il comando **tail** permette di stampare le ultime righe di un file su terminale.

```
$ tail [nomeFile]
$ tail hello.txt
```

Possiamo specificare quante righe stampare:

```
$ tail -n [righeDaStampare] [nomeFile]
$ tail -n 5 hello.txt
```

Questo esempio stampa le ultime 5 righe del file "hello.txt".

2.1.17 wc

Il comando **wc**, ovvero "word count", serve per contare delle specifiche informazioni in un file.

```
$ wc [nomeFile]
$ wc file.txt
```

Il comando restituisce 3 valori in questo ordine:

- Numero di righe
- Numero di parole
- Numero di byte

Il comando può essere eseguito con un filtro passato come attributo:

- **-l** per le righe
- **-w** per le parole
- **-c** per i byte

2.2 Wildcards

Le wildcard permettono di mantenere un aspetto generico di alcuni dati per lavorare su più entità in contemporanea.

Le wildcard quindi funzionano da veri e propri **placeholder**. Questa è una funzionalità della shell, la quale **espande** le wildcard una volta che gli arrivano: sostituisce le wildcard e genera un elenco di tutte le unità che soddisfano la wildcard.

Per esempio:

```
$ ls *.c
"espansione shell"
$ ls file1.c file2.c file3.c
```

Ai comandi non viene mai passata la shell, bensì gli argomenti precisi su cui lavorare.

2.2.1 Asterisco (*)

L'asterisco (*) usato come wildcard permette di specificare una **stringa qualsiasi** di lunghezza qualsiasi, per esempio:

```
$ ls *.c
```

Questo comando stampa tutti i file con estensione ".c", ovvero i file con nome qualsiasi che hanno estensione ".c".

2.2.2 Punto Interrogativo (?)

Il punto interrogativo (?) usato come wildcard indica un **carattere** qualsiasi.

```
$ ls prova?.c
```

Questo comando stampa tutti i file con prefisso "prova", suffisso ".c" e un carattere qualsiasi che divide suffisso e prefisso specificati, in questo caso utile per stampare tutte le prove / test possibilmente nominati "prova1", "prova2" e così via.

2.3 Reindirizzazione

2.3.1 Reindirizzare l'output

Si può reindirizzare lo **standard output** di un comando in questo modo:

```
$ [comando] > [destinazione]
$ ls > lsLog.txt
```

Questo comando scrive in "lsLog.txt" quello che sarebbe dovuto essere stampato sul terminale dal comando "ls".

Attenzione: questo tipo di reindirizzazione **sovrascrive** ogni volta il file intero, se si desidera eseguire un'aggiunta in fondo al file senza perdere i dati precedenti allora bisogna usare l'**append**:

```
$ [comando] >> [destinazione]
$ ls >> lsLog.txt
```

2.3.2 Reindirizzare l'input

Come l'output, si può reindirizzare lo **standard input**:

```
$ [destinazione] < [sorgente]
$ sort < dati.txt
```

In questo esempio, si stampa in ordine alfabetico il contenuto di "dati.txt"

2.3.3 Reindirizzare gli errori

Anche il flusso **standard error** può essere reindirizzato:

```
$ [comando] 2> [destinazione]
$ ls 2> lsLog.txt
```

2.4 Pipe

La **pipe** collega più comandi in questo modo:

stdout del primo comando → stdin del secondo comando

Ecco il template e un esempio:

```
$ [comando1] | [comando2]  
$ ls | less
```

In questo esempio viene eseguito "ls" e viene poi stampato secondo il comando "less".
La pipe evita di creare file intermedi che sarebbero necessari per realizzare il solito effetto.

2.5 Virgolette

Per evitare che non ci sia ambiguità negli argomenti, dato che la shell utilizza gli spazi per separare argomenti, comandi e opzioni, bisogna utilizzare le **virgolette**:

```
$ mkdir "nuova cartellozza"
```

3 Filesystem UNIX

Il **filesystem** è un albero che ha come radice la **root**, indicata da un `"/`

3.1 Path

Il filesystem localizza ogni file e directory tramite un **path**, il quale può essere:

- **Assoluto**: parte dalla root.
- **Relativo**: parte dalla directory corrente.

3.2 Marcature temporali

tenere traccia del tempo in UNIX è importante: i timestamp sono spesso utilizzati per prendere decisioni importanti, come la ricompilazione di un file con un timestamp vecchio. In UNIX ogni file ha 3 marcature temporali:

- **atime**: access time, ultimo accesso.
- **mtime**: modification time, ultima modifica del contenuto.
- **ctime**: change time, ultima modifica dei metadati.

3.3 File nascosti

I **file nascosti** sono i file il cui nome solitamente inizia con il punto.

`.txt .config .bashrc`

4 Programmi UNIX

4.1 Flussi standard

Ogni programma ha 3 flussi standard:

- **standard input**
- **standard output**
- **standard error**

Questi flussi possono essere reindirizzati altrove dal terminale.

5 Linguaggio C

Il linguaggio **C** è un linguaggio compilato, pertanto, prima di eseguire un file bisogna convertirlo in binario.

Esistono diversi standard C: C89, C95, C99, C11, C17, C23.

5.1 Compilatore

Il compilatore più conosciuto e usato per C è **gcc**, ovvero "GNU C Compiler", anche se adesso viene definito come "GNU Compiler Collection" perché diventato un progetto più ampio che include anche altri compilatori come quello di Fortran e C++.

```
gcc hello.c -o hello
```

In questo caso, il comando:

- compila il file "hello.c"
- rinomina l'eseguibile come "hello"

Ogni programma eseguito ritorna un risultato per determinare se è terminato con successo o meno, si ottiene così:

```
$ gcc hello.c
$ hello.c
$ echo $?
0
```

5.1.1 Warnings

I **warnings** sono importanti e non vanno ignorati, per compilare e stamparli tutti si fa così:

```
$ gcc -Wall hello.c
```

Ogni warning contiene un messaggio e la location (riga e carattere) di esso.

5.2 Librerie

Nei programmi in C è possibile utilizzare delle **direttive di preprocessazione** (o del preprocessore) che permettono di importare funzioni e strumenti utili allo svolgimento del programma:

```
#include <stdio.h>
```

5.3 Variabili

In C possiamo usare le **variabili**. Si possono inizializzare ad un valore di un tipo specifico, assegnato al momento della **dichiarazione**.

```
int x;
x = 32;
```

5.4 printf

Questo è un comando base per stampare qualcosa su console:

```
printf(" Hello World ");
```

5.4.1 Placeholders

Per stampare le variabili bisogna utilizzare i **placeholders**:

```
int x = 10;
printf("%d", x)
```

Bisogna scegliere il placeholder corretto per riuscire a stampare una variabile; la scelta va in base al tipo:

- **%d**: numeri interi (int)
- **%i**: numeri interi (int)
- **%f**: numeri decimali (double)
 - Per stampare uno specifico numero di cifre decimali si aggiunge un "." dopo "%" e poi si specifica il numero:

```
printf("%.2f", x)
```

Questa stampa mostrerà per esempio solo 2 cifre decimali. Usato sugli altri placeholder questo provoca l'ingrandimento di padding a destra (numero negativo) o sinistra (numero positivo) di ciò che viene stampato.

- **%c**: caratteri (char)
- **%s**: stringa
- **%u**: numeri interi senza segno (unsigned int)
- **%x**: numero intero in esadecimale
- **%p**: indirizzo di memoria, puntatore

5.5 Commenti

I commenti in C possono essere inseriti in vari modi:

- Commenti su una riga: aperti da `//`, non vengono chiusi
- Commenti su più righe: delimitati da `/*` e da `*/`.

6 Esercizi

6.1 Esercizio 1

Partendo dalla propria home, svolgere queste operazioni dal terminale LINUX:

1. Creare una directory lezione02
2. Entrarci
3. Creare tre file vuoti: "a.c", "b.c", "note.txt"
4. Visualizzare solamente i file con estensione ".c"
5. Salvare quell'elenco in "sorgenti.txt"
6. Aggiungere il contenuto di "sorgenti.txt" a "note.txt"
7. Contare quante righe contiene "note.txt"

Soluzione:

```
$ cd ~  
$ mkdir lezione02  
$ cd lezione02  
$ touch a.c b.c node.txt  
$ ls *.c > sorgenti.txt  
$ sorgenti.txt >> note.txt  
$ qw -l note.txt
```